

Theoretical Computer Science Exam, June 26, 2019

The exam consists of **4 exercises**. Available time: 1 hr 40 min.; you may write your answers in Italian, if you wish.

All exercises must be addressed for the written exam to be considered sufficient.

F.NAME L.NAME MATRICOLA

I have passed the midterm exam and I intend to skip the first part (Ex.1-2) (select one): **YES NO**

Exercise 1 (8 pts)

1. Design a least general grammar that generates the language $L = L_1^*$, where $L_1 = \{ xx^R \mid x \in \{a, b\}^* \}$
2. Write a derivation for the string $aabbaa \in L$ according to the above defined grammar.
3. Design a least powerful automaton that accepts language L defined above.
4. Write a computation of the above automaton that accepts the string $aabb \in L$

Exercise 2 (8 pts)

1. Write a Monadic First Order (or Monadic Second Order if necessary) logic formula that characterizes the language of nonempty strings over the alphabet $I = \{a, b\}$, such that the total number of a 's included in the string is even.
2. Write a Monadic First Order (or Monadic Second Order if necessary) logic formula that characterizes the language of nonempty strings over the alphabet $I = \{a, b\}$, such that a 's occur only at positions with an even index value (remember that in the word structure the index of position starts with the value 0).
3. Write a Monadic First Order (or Monadic Second Order if necessary) logic formula that characterizes the language of nonempty strings over the alphabet $I = \{a, b\}$, such that it cannot happen that the j -th occurrence of an a symbol, with j odd (that is, the first, third, fifth, ... occurrence) occurs at a position whose index is even.

Exercise 3 (8 pts)

Consider the following language L over the binary alphabet $\Sigma = \{0, 1\}$:

$$L = \{ x \mid x \in \Sigma^* \text{ and } x \text{ is the binary encoding of a positive integer that is divisible by three} \}$$

Answer the following questions, providing suitable clear and precise justifications.

1. Is the language L decidable? Is it semidecidable?
2. Is it possible to build a Turing Machine that enumerates L ?
3. Given a generic Turing machine m , is it possible to decide whether m accepts L ?
4. Given a generic Turing machine m , is it possible to decide whether m enumerates L ?
5. Consider the following set S

$$S = \{ x \mid \forall n (f_x(n) \neq \perp \rightarrow f_x(n) = 3 \cdot n) \}$$

Describe in words the set S and argue whether S is decidable or semidecidable.

Exercise 4 (8 pts)

Design, or at least sketch in a clear, sufficiently detailed way, a Turing machine that recognizes strings of the following kind:

$$a^n b a^{n-1} b \dots a^2 b a b b \text{ with } n > 1$$

an example of an acceptable input string is *aaaabaaabaababb*.

The time complexity of the Turing machine should be as reduced as possible.

The pseudocode below encodes an algorithm (to be executed by the RAM) to recognize the strings described above.

```
-- read first sequence of type  $a^i b$ 
m = 0;
do
    read(c);  if ( c == 'a' ) then m = m+1; endif;
while ( c == 'a' );
if ( c == 'b' and m > 0 ) then mPrev = m; else write('no'); halt; endif;
do
    -- read next sequence of type  $a^j b$ 
    m = 0;
    do
        read(c);  if ( c == 'a' ) then m = m + 1;  endif;
    while ( c == 'a' );
    if ( c == 'b' and m == mPrev - 1 ) then mPrev = m;  else write('no'); halt; endif;
while ( m > 0 );
write('yes'); halt;
```

Evaluate in a precise way the time complexity class of the above program in the case of an accepted string, with both the uniform and the logarithmic cost criteria.

Compare the complexity of the Turing and RAM machines when the logarithmic time criterion is adopted.

Solutions

Exercise 1

1. The language strings are possibly empty lists of possibly empty palindromes

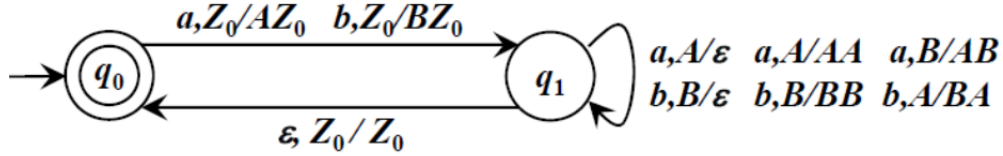
$$S \rightarrow SP \mid \varepsilon$$

$$P \rightarrow aPa \mid bPb \mid \varepsilon$$

2. Here is the derivation. It is interesting to notice that the string admits other derivations.

$$S \Rightarrow^3 SP PP \Rightarrow PPP \Rightarrow^3 aPabPbaPa \Rightarrow^3 aabb aa$$

3. A nondeterministic PDA is necessary



4. Here is the computation. Since the automaton is nondeterministic, other computations are possible.

$$\begin{aligned} < q_0, aabb, Z_0 > \vdash < q_1, abb, AZ_0 > \vdash < q_1, bb, Z_0 > \vdash < q_0, bb, Z_0 > \vdash < q_1, b, BZ_0 > \vdash < q_1, \varepsilon, Z_0 > \vdash \\ & \vdash < q_0, \varepsilon, Z_0 > \end{aligned}$$

Exercise 2

We introduce the predicate $E(x)$ meaning that the number of a 's up to and including position x is even

$$\begin{aligned} \exists E \big((E(0) \leftrightarrow \neg a(0)) \wedge \forall x (\neg \text{last}(x) \rightarrow (a(x+1) \rightarrow (E(x+1) \leftrightarrow \neg E(x)) \wedge \\ (\neg a(x+1) \rightarrow (E(x+1) \leftrightarrow E(x))) \\) \\) \end{aligned}$$

We introduce the predicate $X(x)$ meaning that position x is even

$$\exists X \forall z (X(0) \wedge (\neg \text{last}(z) \rightarrow (X(z) \leftrightarrow \neg X(z+1))))$$

Then

$$\begin{aligned} 1. \\ \exists E \big((E(0) \leftrightarrow \neg a(0)) \wedge \forall x (\neg \text{last}(x) \rightarrow (a(x+1) \rightarrow (E(x+1) \leftrightarrow \neg E(x)) \wedge \\ (\neg a(x+1) \rightarrow (E(x+1) \leftrightarrow E(x))) \\) \wedge \\ \forall y (\text{last}(y) \rightarrow E(y)) \\) \end{aligned}$$

$$2. \\ \exists X (\forall z (X(0) \wedge (\neg \text{last}(z) \rightarrow (X(z) \leftrightarrow \neg X(z+1)))) \wedge \forall y (a(y) \rightarrow X(y)))$$

3. Combine the above definitions of predicates E and X with the formula $\neg \exists x (a(x) \wedge \neg E(x) \wedge X(x))$

$$\begin{aligned} \exists E \exists X \big((E(0) \leftrightarrow \neg a(0)) \wedge \forall x (\neg \text{last}(x) \rightarrow (a(x+1) \rightarrow (E(x+1) \leftrightarrow \neg E(x)) \wedge \\ (\neg a(x+1) \rightarrow (E(x+1) \leftrightarrow E(x))) \\) \\) \\ \wedge \\ (\forall z (X(0) \wedge (\neg \text{last}(z) \rightarrow (X(z) \leftrightarrow \neg X(z+1)))) \\ \wedge \\ \neg \exists v (a(v) \wedge \neg E(v) \wedge X(v)) \\) \end{aligned}$$

Exercise 3

1. Language L is obviously decidable: it is an easy programming exercise to write a program that checks that a given binary string encodes a number that is a multiple of 3. Being decidable, L is also semidecidable.
2. Yes, for the same reason as in the first point above.
3. No, based on the Rice theorem.
4. No, based on the Rice theorem: enumerating a set is still a way to compute a function.
5. Set S includes exactly the computable functions that, where defined, return the n -th multiple of their input parameter n . S is undecidable due to the Rice theorem, and its complement, $\neg S = \{ x \mid \exists n (f_x(n) = \perp \wedge f_x(n) \neq 3 \cdot n) \}$ is semidecidable (using the customary dovetailing simulation technique), therefore S is not semidecidable.

Exercise 4

Solution

A 1-tape Turing machine can accept the correct strings in linear time (actually in real-time) by writing on the memory tape a sequence of symbols of length equal to the number of a 's read before the first b . The memory tape is then used as a counter (in unary notation), by deleting one of the symbols for every successive sequence of a 's, to check that they are of decreasing length.

The time complexity of the RAM machine, adopting the uniform cost criterion, is $\Theta(|x|)$, i.e., it is linear in the length of the input string x .

Notice that $|x|$ is $\Theta(n^2)$.

When the logarithmic cost criterion is applied, then the dominating complexity factor is the one concerning the execution of the " $m = m + 1$;" instruction in the inner loop. This instruction is translated into the following RAM code, with the complexity figure shown on the right, where p is the memory position of the cell storing the value of the program variable m .

LOAD	p	$l(p) + l(M[p])$
ADD=	1	$l(M[0]) + l(1)$
STORE	p	$l(p) + l(M[p])$

If we call i the value of variable m before the increment, then the complexity of these instructions is

$$2 l(p) + 2 l(i) + l(i+1) \cong 2 l(p) + 3 l(i)$$

This is executed for each a symbol in the input, hence the overall complexity is

$$\sum_{k=1}^n \sum_{i=1}^k (2l(p) + 3l(i)) = \sum_{k=1}^n (2k l(p) + 3 \sum_{i=1}^k l(i)) \cong 2l(p) \frac{n(n+1)}{2} + 3 \sum_{k=1}^n (k \log(k) - k)$$

which is $\Theta(n^2 \log n) = \Theta(|x| \log \sqrt{|x|})$, $|x|$ being the length of the input string. The time complexity of the RAM machine is therefore worse than that of the Turing machine.